

NGINX Cheat Sheet & Guía de Arquitectura

Manual de Referencia Rápida para Desarrolladores, Administradores y Arquitectos de Software

1. ARQUITECTURA Y MODELO DE PROCESOS

NGINX es un servidor web y proxy inverso de alto rendimiento diseñado para manejar grandes cantidades de conexiones concurrentes de forma eficiente. A diferencia de servidores tradicionales como Apache (basados en procesos o hilos), NGINX utiliza un **modelo asíncrono y basado en eventos (epoll en Linux)**, lo que lo hace extremadamente ligero en consumo de CPU/RAM y adecuado para arquitecturas de microservicios.

Componente / Proceso	Descripción y Función Arquitectónica
Master Process	Encargado de leer y evaluar ficheros de configuración, abrir sockets de red y gestionar el ciclo de vida de los procesos trabajadores.
Worker Processes	Manejan las conexiones entrantes de clientes en paralelo y de forma no bloqueante.
Módulos Estáticos	Funcionalidades cargadas de forma estática durante la compilación.
Rutas Principales	• /etc/nginx/nginx.conf (Config global) • /etc/nginx/sites-available/ y sites-enabled/ (Virtual hosts Debian/Ubuntu) • /var/log/nginx/access.log y error.log (Logs del sistema)

2. VIRTUAL HOSTS (IP Y NOMBRE)

Virtual Host Basado en Nombre (Recomendado): Permite alojar múltiples dominios en una sola dirección IP.

Ejemplo: Virtual Host por Nombre en puerto 80

```
server {
    listen 80; listen [::]:80;
    server_name mi-web.com www.mi-web.com;
    root /var/www/mi-web.com/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Virtual Host Basado en IP: Utilizado cuando el servidor dispone de múltiples direcciones IP públicas o privadas.

Ejemplo: Virtual Host asignado a IP 192.0.2.7

```
server {
    listen 192.0.2.7:80;
    server_name mi-web.com;
    root /var/www/mi-web/html;
    index index.html;
    location / { try_files $uri $uri/ =404; }
}
```

3. HTTPS, SNI Y REDIRECCIONES

Configuración segura con cifrado SSL/TLS. Gracias al soporte de **SNI (Server Name Indication)**, NGINX puede presentar diferentes certificados SSL según el hostname solicitado compartiendo una única dirección IP, sin requerir configuración especial.

HTTPS con Redirección 301 y TLS 1.2/1.3

```
# Redirección global HTTP -> HTTPS (Puerto 80)
server {
    listen 80; server_name sitio1.com www.sitio1.com;
    return 301 https://$host$request_uri;
}

# Bloque Servidor Seguro (Puerto 443 SSL)
server {
    listen 443 ssl;
    server_name www.mi-web.com;
    ssl_certificate      /etc/nginx/certs/www.mi-web.com.crt;
    ssl_certificate_key  /etc/nginx/certs/www.mi-web.com.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers          HIGH:!aNULL:!MD5;
}
```

Mecanismos de Redirección:

Directiva	Sintaxis / Ejemplo	Uso y Comportamiento
return	return 301 https://nuevo.com\$request_uri;	Respuesta directa y simple con código de estado (301 permanente, 302 temporal).
rewrite	rewrite ^/old/(.*)\$ /new/\$1 permanent;	Reescritura avanzada basada en Regex. Flags: permanent (301), redirect (302), last, break.

4. PROXY INVERSO Y BALANCEO DE CARGA (UPSTREAM)

NGINX actúa como intermediario recibiendo peticiones de clientes y reenviándolas a servidores backend.

Proxy Inverso con Balanceo de Carga en Clúster

```
upstream backendcluster {
    server backend1.ejemplo.com weight=5;
    server backend2.ejemplo.com;
    server 192.0.2.50:3000 max_fails=3 fail_timeout=30s;
}

server {
    listen 80; server_name app.ejemplo.com;
    location / {
        proxy_pass http://backendcluster;
    }
}
```

Parámetros de Ajuste del Cluster Upstream:

- **weight=N:** Asigna más peso relativo a nodos más potentes para recibir mayor proporción de tráfico.
- **max_fails / fail_timeout:** Define el número de fallos tras los cuales un nodo se considera inactivo durante el tiempo especificado.
- **max_conns=N:** Limita el número máximo de conexiones simultáneas por servidor backend.

5. INTEGRACIÓN CON RUNTIMES DE APLICACIÓN

Tecnología	Directiva Principal	Detalles de Configuración y Protocolo
PHP (FPM)	fastcgi_pass	Conecta con el socket Unix o de red de PHP-FPM. Ejemplo: fastcgi_pass unix:/var/run/php/php8.2-fpm.sock; Requiere include fastcgi_params; y fastcgi_param SCRIPT_FILENAME \$document_root\$fastcgi_script_name;
Python	uwsgi_pass	Interfaz WSGI utilizando el protocolo binario uwsgi optimizado. Ejemplo: uwsgi_pass unix:///var/run/uwsgi-app.sock; con include uwsgi_params;
Java (Tomcat)	proxy_pass o ajp_pass	1. Vía HTTP Estándar: proxy_pass http://tomcat:8080; 2. Vía AJP (puerto 8009): Requiere compilar el módulo de terceros nginx_ajp_module para habilitar ajp_pass 127.0.0.1:8009;

6. SEGURIDAD, CONTROL DE ACCESO Y RATE LIMITING

Filtrado por IP y Autenticación HTTP Básica

```
# 1. Control de Acceso por IP (Evaluación en orden estricto)
location /admin/ {
    allow 192.168.1.10; allow 10.0.0.0/24;
    deny all; # Se detiene en la primera coincidencia
}

# 2. Autenticación HTTP Básica
location /privado/ {
    auth_basic "Area Restringida";
    auth_basic_user_file /etc/nginx/.htpasswd; # Generado con htpasswd
}
```

Limitación de Recursos y Rate Limiting (Protección Anti-Abuso):

- **limit_conn_zone / limit_conn:** Limita el número de conexiones simultáneas por cliente IP (ej. 5 conexiones). Excederlo retorna código **503**.
- **limit_req_zone / limit_req:** Limita la tasa de peticiones (ej. 5 req/s por IP con ráfaga de 10). Excederlo retorna código **503**.
- **client_max_body_size:** Ajusta el tamaño máximo permitido para subidas de archivos (por defecto 1 MiB).

7. TUNING DE RENDIMIENTO Y CONCURRENCIA

Directiva	Valor por Defecto / Sugerido	Impacto y Optimización
worker_processes	auto (o N° de núcleos CPU)	Define cuántos procesos worker se ejecutan. Más procesos que núcleos suele ser ineficiente salvo I/O intensivo.
worker_connections	1024 (aumentar a 8192+)	Conexiones máximas por worker. Máx Conexiones Totales = worker_processes x worker_connections.
multi_accept	on	Permite que un worker acepte múltiples conexiones pendientes simultáneamente.
keepalive_requests	100	Número máximo de peticiones permitidas por una misma conexión antes de cerrarla.

keepalive_timeout	65s	Tiempo de espera para conexiones inactivas. Valores bajos liberan recursos; altos evitan reconexiones.
ssl_session_cache	shared:SSL:10m;	Guarda sesiones TLS en memoria compartida reduciendo la sobrecarga de handshakes.